

CSE 144 Final Project Report

Transfer Learning Challenge

Skye Gunasekaran (CruzID: 195677)

Spring 2026

1 Introduction

The goal of this project is to build an image classifier for the CSE 144 transfer learning challenge. The task contains 100 target classes, but only a small number of labeled examples per class are available, making it difficult to train a high-capacity convolutional or transformer model from scratch. The model must predict a class label for each test image and export the predictions in the Kaggle submission format.

Transfer learning is appropriate because the labeled training set is small relative to the complexity of the visual recognition task. Instead of learning all image features from scratch, the submitted pipeline uses a pretrained vision transformer backbone and trains a lightweight classifier head on top of the pretrained representation. This approach should improve generalization because the backbone already encodes useful visual features such as shapes, textures, object parts, and global image structure.

My final approach used a pretrained DINOv3 ViT-B/16 backbone from HuggingFace with the backbone frozen and a small linear classifier head trained for the 100-way classification task. The main result from the loss curves is that the model learned quickly during the first few epochs and then converged to a stable validation loss around 1.2 cross-entropy. The final Kaggle score should be inserted once the final `submission.csv` is uploaded.

2 Dataset

The dataset is organized into a labeled training directory and an unlabeled test directory. The training code assumes that labels are encoded directly by folder names, so folder `train/0/` maps to class label 0, folder `train/1/` maps to class label 1, and so on through folder `train/99/`. This numeric sorting step is important because Kaggle expects the predicted labels to match the folder numbering.

Table 1: Dataset split used by the training script. Replace the approximate counts if your local dataset differs.

Item	Value
Number of classes	100
Approximate labeled examples	~1000 total, about 10 per class
Validation split	15% of labeled training data
Effective training split	~850 images
Effective validation split	~150 images
Test set	1000 images named 0.jpg through 999.jpg

All images are loaded as RGB images. For training, images are resized and augmented before normalization. The augmentation pipeline includes random horizontal flips, occasional vertical flips, random rotations, random affine translation and scale, color jitter, rare grayscale conversion, and random resized cropping. For validation and test inference, the transform is deterministic: resize to 224×224 , convert to tensor, and normalize using ImageNet mean and standard deviation. This separation ensures that validation loss measures performance on stable inputs rather than randomly augmented versions of the validation images.

3 Implementation

3.1 Model

The final model is a DINOv3 ViT-B/16 classifier. The pretrained backbone is loaded with the HuggingFace model name `facebook/dinov3-vitb16-pretrain-lvd1689m`. The classifier uses the transformer [CLS] token representation as the image embedding. A small task-specific head is attached:

$$\text{logits} = W \cdot \text{LayerNorm}(h_{\text{CLS}}) + b, \quad (1)$$

where $h_{\text{CLS}} \in \mathbb{R}^{768}$ is the hidden representation from the DINOv3 backbone and the output has 100 logits, one for each class.

The backbone was frozen during training, so only the classifier head parameters were updated. This was a practical strategy for a small dataset because it reduced the number of trainable parameters, lowered the risk of overfitting, and made training faster. A future extension would be to unfreeze the last few transformer blocks after the head converges and continue training with a much smaller learning rate.

3.2 Training

The training script uses cross-entropy loss with label smoothing, AdamW optimization, cosine annealing learning-rate scheduling, mixed precision on CUDA, and gradient clipping. The default hyperparameters are summarized in Table 2.

Table 2: Training hyperparameters from the provided training script.

Hyperparameter	Value
Backbone	DINOv3 ViT-B/16
Image size	224×224
Loss	Cross-entropy with label smoothing = 0.1
Optimizer	AdamW
Head learning rate	10^{-3}
Weight decay	0
Scheduler	Cosine annealing, $\eta_{\min} = 10^{-6}$
Epochs	20
Early stopping patience	10 epochs
Batch size	128
Validation split	0.15
Gradient clipping	Max norm 1.0
Random seed	42

The code automatically uses CUDA if available, MPS on Apple Silicon if CUDA is unavailable, and CPU otherwise. When CUDA is used, mixed precision is enabled through `torch.amp.GradScaler`, and matrix multiplication precision is set to `high`. The script saves the model checkpoint with the best validation accuracy and then uses that checkpoint for test-set inference.

4 Experiments

The baseline experiment was a frozen pretrained transformer backbone with a linear classification head. This baseline is strong for the assignment setting because it avoids fitting millions of backbone parameters on a very small training set. The validation split was sampled with a fixed random seed to make the comparison repeatable.

The main tuning choices were the backbone, augmentation policy, learning rate, and freezing strategy. I used stronger augmentation than a simple resize-and-crop baseline because each class has very few examples. Geometric and color augmentations encourage invariance to small viewpoint, position, scale, brightness, and saturation changes. I also used label smoothing to make the classifier less overconfident, which can improve validation behavior when the training set is small.

Recommended ablations to add before final submission are:

1. Compare frozen DINOv3 against a simpler ImageNet-pretrained ResNet baseline.
2. Compare the current augmentation pipeline against a lighter transform with only resize, crop, and horizontal flip.
3. Fine-tune the last transformer block after head-only training using a learning rate around 10^{-5} .
4. Try small nonzero weight decay values such as 10^{-4} or 10^{-3} .

5 Results

Figure 1 shows the training and validation cross-entropy loss over 20 epochs. The training loss drops rapidly from roughly 4.9 to near 1.5 in the first 25 optimizer steps and then gradually decreases

toward about 1.0. The validation loss also improves quickly, decreasing from around 3.1 after the first epoch to about 1.3 by epoch 5. After that, validation loss plateaus near 1.2.

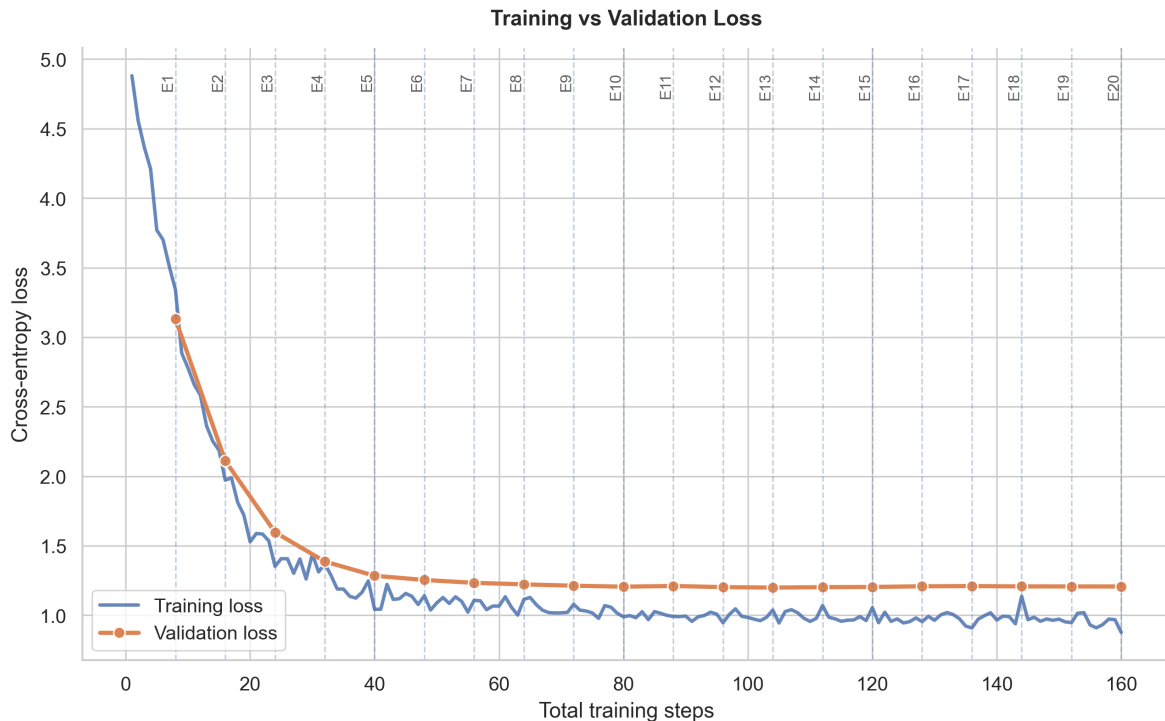


Figure 1: Training and validation loss over total optimizer steps. Validation loss is logged once per epoch.

The curve suggests that the model is not severely overfitting. The validation loss remains above the training loss, but the gap is modest and stable after convergence. The plateau after roughly epochs 8–10 suggests that the frozen backbone and linear head may have extracted most of the easily available signal. Further improvement may require partial fine-tuning of the backbone, additional test-time augmentation, or a small ensemble.

Table 3: Final metrics to fill in after the final run and Kaggle upload.

Metric	Value
Best validation accuracy	0.882
Best validation loss	1.1988
Kaggle public leaderboard score	19
Kaggle submission file	<code>submission.csv</code>

6 Discussion

The most effective part of the approach was using a strong self-supervised pretrained vision transformer. Because the training set is small, the pretrained DINOv3 representation provides much better visual features than a model trained from random initialization. Freezing the backbone also made the experiment stable and computationally efficient.

The main limitation is that only the linear head is trained. This makes the model robust against overfitting, but it may also underfit if the target classes differ from the distribution learned by the pretrained backbone. The validation curve plateau supports this interpretation: training continues to make small improvements, but validation loss stops improving significantly. A second-stage fine-tuning run could improve class-specific features, especially if the last transformer blocks are unfrozen carefully with a low learning rate.

Another limitation is the small validation split. With only about 150 validation examples spread over 100 classes, the validation accuracy may be noisy. Some classes may have only one or two validation examples. Stratified splitting or cross-validation would give a more reliable estimate of generalization, but the current split is a reasonable lightweight validation strategy for fast iteration.

Concrete next steps are to run a partial fine-tuning experiment, add a confusion-matrix or per-class error analysis, and inspect incorrectly classified validation examples. It would also be useful to test whether the vertical flip augmentation helps or hurts, since vertical flips can be unrealistic for some natural image categories.

7 Reproducibility

The code sets the Python, NumPy, and PyTorch random seeds to 42. The validation split is also generated with a PyTorch generator seeded with 42. Deterministic cuDNN behavior is disabled by default for speed, while cuDNN benchmarking is enabled when determinism is off.

To train the model and produce a Kaggle submission, run:

```
python train.py --data_dir /path/to/dataset --mode train
```

This trains the classifier head, saves the best checkpoint to `best_model.pth`, saves the loss plot to `loss_curve.png`, and writes `submission.csv`. To run inference from an existing checkpoint, run:

```
python train.py --data_dir /path/to/dataset --mode infer \
  --checkpoint best_model.pth \
  --output_csv submission.csv
```

The main Python dependencies are PyTorch, torchvision, transformers, pillow, numpy, pandas, matplotlib, and seaborn. Add the exact package versions from the final environment using:

```
pip freeze > requirements.txt
```

8 Team Contributions

1. Skye Gunasekaran: finished project from end-to-end.

9 References

1. Meta AI. DINO / DINOv2 / DINOv3 self-supervised vision transformer model family and pretrained weights.
2. HuggingFace Transformers documentation: <https://huggingface.co/docs/transformers/index>.
3. PyTorch documentation: <https://pytorch.org/docs/stable/index.html>.
4. torchvision transforms documentation: <https://pytorch.org/vision/stable/transforms.html>.